Install required Python packages

python3.11 -m pip install pandas dash

```
(11.3 MB)
   ━━━━━━━━━━━━━━━━━━ 11.3/11.3 MB 146.4 MB/s  0:00:00
Downloading dash-4.3.0-py3-none-any.whl (8.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━ 8.4/8.4 MB 96.9 MB/s  0:00:00
Downloading mcp-1.28.0-py3-none-any.whl (221 kB)
Downloading httpx_sse-0.4.3-py3-none-any.whl (9.0 kB)
Downloading numpy-2.4.6-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (
16.9 MB)
   ━━━━━━━━━━━━━━━━━━ 16.9/16.9 MB 181.9 MB/s  0:00:00
Downloading plotly-6.8.0-py3-none-any.whl (9.9 MB)
   ━━━━━━━━━━━━━━━━━━━━ 9.9/9.9 MB 164.2 MB/s  0:00:00
Downloading narwhals-2.22.1-py3-none-any.whl (454 kB)
Downloading pydantic_settings-2.14.2-py3-none-any.whl (61 kB)
Downloading sse_starlette-3.4.5-py3-none-any.whl (16 kB)
Downloading starlette-1.3.1-py3-none-any.whl (73 kB)
Downloading retrying-1.4.2-py3-none-any.whl (10 kB)
Installing collected packages: retrying, numpy, narwhals, httpx-sse, starlette, plotl
y, pandas, sse-starlette, pydantic-settings, mcp, dash
```

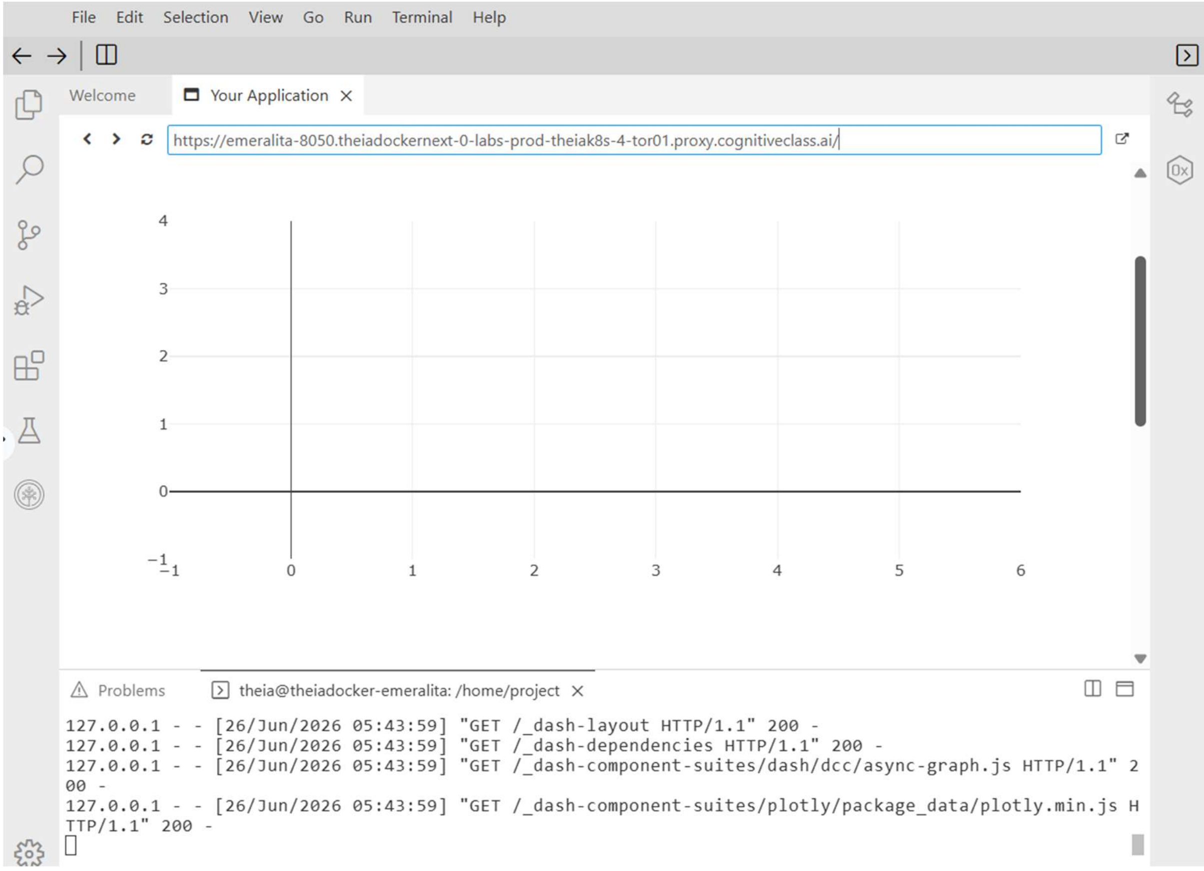
Download a skeleton dashboard application and dataset

wget "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/spacex_launch_dash.csv"

```
theia@theiadocker-emeralita:/home/project$ wget "https://cf-courses-data.s3.us.cloud-
object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/spacex_launch_dash
.csv"
--2026-06-26 05:38:24--  https://cf-courses-data.s3.us.cloud-object-storage.appdomain
.cloud/IBM-DS0321EN-SkillsNetwork/datasets/spacex_launch_dash.csv
Resolving cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud (cf-courses-data
.s3.us.cloud-object-storage.appdomain.cloud)... 169.63.118.104, 169.63.118.104
Connecting to cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud (cf-courses-
data.s3.us.cloud-object-storage.appdomain.cloud)|169.63.118.104|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2476 (2.4K) [text/csv]
Saving to: 'spacex_launch_dash.csv'

spacex_launch 100%   2.42K  --.-KB/s    in 0s

2026-06-26 05:38:24 (702 MB/s) - 'spacex_launch_dash.csv' saved [2476/2476]
```

wget "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/t4-Vy4iOU19i8y6E3Px_ww/spacex-dash-app.py"

```
theia@theiadocker-emeralita:/home/project$ wget "https://cf-courses-data.s3.us.cloud-
object-storage.appdomain.cloud/t4-Vy4iOU19i8y6E3Px_ww/spacex-dash-app.py"
--2026-06-26 05:42:12--  https://cf-courses-data.s3.us.cloud-object-storage.appdomain
.cloud/t4-Vy4iOU19i8y6E3Px_ww/spacex-dash-app.py
Resolving cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud (cf-courses-data
.s3.us.cloud-object-storage.appdomain.cloud)... 169.63.118.104, 169.63.118.104
Connecting to cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud (cf-courses-
data.s3.us.cloud-object-storage.appdomain.cloud)|169.63.118.104|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2075 (2.0K) [application/x-python]
Saving to: 'spacex-dash-app.py'

spacex-dash-app.py    100%[======================>]   2.03K  --.-KB/s    in 0s

2026-06-26 05:42:12 (284 MB/s) - 'spacex-dash-app.py' saved [2075/2075]
```



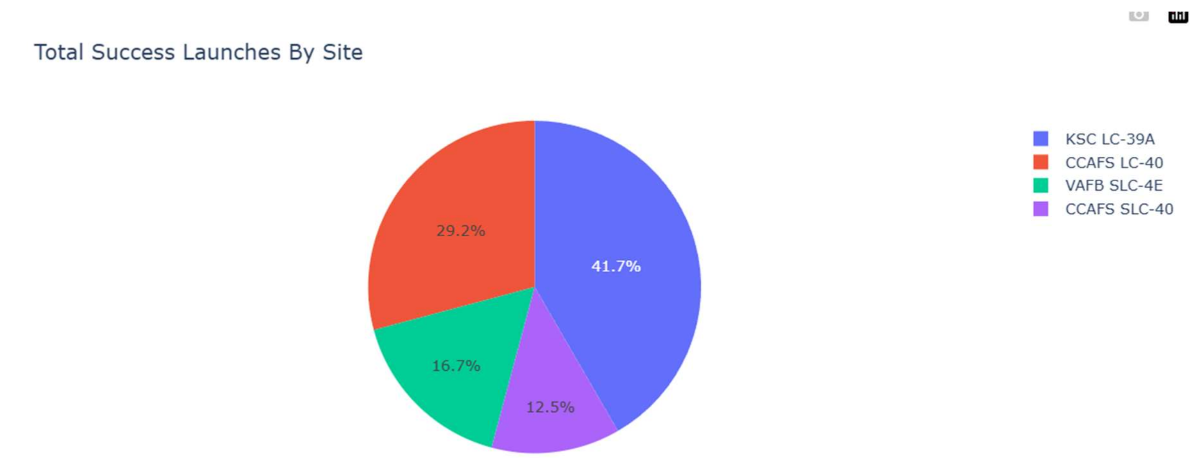

```python
# Import required libraries
import pandas as pd
import dash
from dash import html
from dash import dcc
from dash.dependencies import Input, Output
import plotly.express as px

# Read the airline data into pandas dataframe
spacex_df = pd.read_csv("spacex_launch_dash.csv")
max_payload = spacex_df['Payload Mass (kg)'].max()
min_payload = spacex_df['Payload Mass (kg)'].min()

# Create a dash application
app = dash.Dash(__name__)

# Create an app layout
app.layout = html.Div(children=[
    html.H1('SpaceX Launch Records Dashboard',
            style={'textAlign': 'center', 'color': '#503D36', 'font-size':
40}),
```

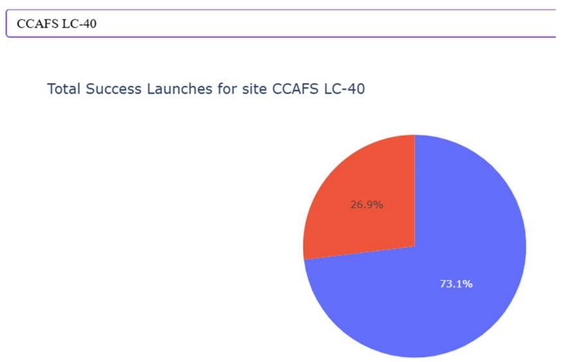
```
    # TASK 1: Add a Launch Site Drop-down Input Component
    dcc.Dropdown(
        id='site-dropdown',
        options=[
            {'label': 'All Sites', 'value': 'ALL'},
            {'label': 'CCAFS LC-40', 'value': 'CCAFS LC-40'},
            {'label': 'VAFB SLC-4E', 'value': 'VAFB SLC-4E'},
            {'label': 'KSC LC-39A', 'value': 'KSC LC-39A'},
            {'label': 'CCAFS SLC-40', 'value': 'CCAFS SLC-40'}
        ],
        value='ALL',
        placeholder="Select a Launch Site here",
        searchable=True
    ),
    html.Br(),

    # TASK 2: Add a pie chart to show the total successful launches count for
all sites
    html.Div(dcc.Graph(id='success-pie-chart')),
    html.Br(),

    html.P("Payload range (Kg):"),

    # TASK 3: Add a slider to select payload range
    dcc.RangeSlider(
        id='payload-slider',
        min=0,
        max=10000,
        step=1000,
        marks={0: '0', 2500: '2500', 5000: '5000', 7500: '7500', 10000:
'10000'},
        value=[min_payload, max_payload]
    ),
    html.Br(),

    # TASK 4: Add a scatter chart to show the correlation between payload and
launch success
    html.Div(dcc.Graph(id='success-payload-scatter-chart')),
])

# TASK 2: Callback function for `site-dropdown` as input, `success-pie-chart`
as output
@app.callback(
    Output(component_id='success-pie-chart', component_property='figure'),
    Input(component_id='site-dropdown', component_property='value')
)
def get_pie_chart(entered_site):
    if entered_site == 'ALL':
```

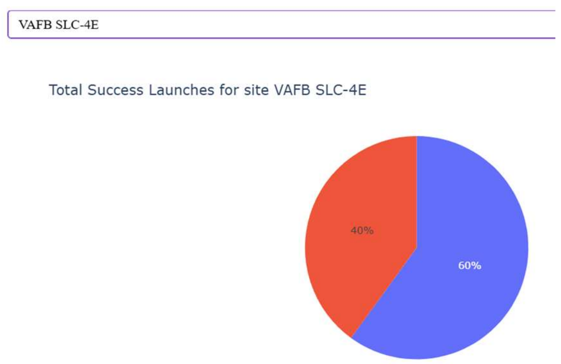
```
        fig = px.pie(
            spacex_df,
            values='class',
            names='Launch Site',
            title='Total Success Launches By Site'
        )
        return fig
    else:
        filtered_df = spacex_df[spacex_df['Launch Site'] == entered_site]
        df_counts = filtered_df['class'].value_counts().reset_index()
        df_counts.columns = ['class', 'count']

        fig = px.pie(
            df_counts,
            values='count',
            names='class',
            title=f'Total Success Launches for site {entered_site}'
        )
        return fig

# TASK 4: Callback function for `site-dropdown` and `payload-slider` as
inputs, `success-payload-scatter-chart` as output
@app.callback(
    Output(component_id='success-payload-scatter-chart',
component_property='figure'),
    [
        Input(component_id='site-dropdown', component_property='value'),
        Input(component_id='payload-slider', component_property='value')
    ]
)
def get_scatter_chart(entered_site, payload_range):
    low, high = payload_range
    mask = (spacex_df['Payload Mass (kg)'] >= low) & (spacex_df['Payload Mass
(kg)'] <= high)
    filtered_df = spacex_df[mask]

    if entered_site == 'ALL':
        fig = px.scatter(
            filtered_df,
            x='Payload Mass (kg)',
            y='class',
            color='Booster Version Category',
            title='Correlation between Payload and Success for all Sites'
        )
        return fig
    else:
        site_filtered_df = filtered_df[filtered_df['Launch Site'] ==
entered_site]
```

```
        fig = px.scatter(
            site_filtered_df,
            x='Payload Mass (kg)',
            y='class',
            color='Booster Version Category',
            title=f'Correlation between Payload and Success for site
{entered_site}'
        )
        return fig


if __name__ == '__main__':
    app.run(port=8051)
```

python3.11 spacex-dash-app.py







1. Which site has the largest successful launches?
   KSC LC-39A
   Select the "All Sites" option from the dropdown, the Pie Chart shows that KSC LC-39A has the largest slice/percentage of successful launches (class = 1) compared to the other sites.
2. Which site has the highest launch success rate?
   KSC LC-39A
   Filter the dropdown to view each site individually, KSC LC-39A displays the highest ratio of success (1) versus failure (0), achieving a success rate of approximately 76.9%.
3. Which payload range(s) has the highest launch success rate?
   2,000 kg - 4,000 kg
   On the Scatter Plot, looking closely at the payload range between 2,000 and 4,000 kg reveals a dense cluster of successful points (class = 1) with very few failures in that specific interval.
4. Which payload range(s) has the lowest launch success rate?
   Above 5,000 kg (specifically between 5,400 kg - 10,000 kg)
   When looking at heavier payloads on the Scatter Plot, most of the data points drop to the bottom line (class = 0), indicating that missions with very heavy payloads experienced a higher rate of failure in this dataset.

5. Which F9 Booster version has the highest launch success rate?
   B5
   In the Scatter Plot legend, different booster versions are color-coded. The points representing the B5 version are located exclusively on the top success line (class = 1), showing a perfect 100% success record in this data. The FT (Full Thrust) version is also highly successful.